НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОСТРОЗЬКА АКАДЕМІЯ»
Кафедра інформаційних технологій та аналітики даних

Лабораторна робота №2. Основи CSS: Box Model, Cascade, Specificity та AI-assisted верстка

з дисципліни «Сучасна веброзробка: HTML, CSS, JS/TS»

1. Мета роботи

Дослідити фундаментальні механізми роботи CSS (Box Model, Cascade, Specificity, Inheritance) та навчитися свідомо застосовувати їх при розробці вебсторінок. Опанувати підхід AI-assisted розробки, зберігаючи контроль над архітектурою та якістю коду.

2. Цілі та результати навчання

Після виконання лабораторної роботи студент повинен:

- Розуміти та вміти пояснити, як формуються розміри елементів (Box Model).
- Знати принципи вирішення конфліктів стилів (Cascade та Specificity).
- Розуміти механізм успадкування властивостей (Inheritance).
- Вміти використовувати CSS Variables для створення консистентного дизайну.
- Володіти інструментами розробника (DevTools) для дебагінгу CSS.
- Вміти ефективно використовувати AI-асистентів для генерації розмітки та стилів, здатних критично оцінювати та рефакторити згенерований код.

3. Завдання лабораторної роботи

1. **Провести серію досліджень** базових концепцій CSS, спираючись на матеріали web.dev та експерименти в браузері.
2. **Виконати AI-assisted реалізацію макету**, застосувавши досліджені принципи на практиці. Макет і вимоги до нього описані в окремому документі — технічне завдання «Секція тарифних планів».
3. **Оформити звіт-дослідження**, який доводить розуміння матеріалу на основі власних спостережень та DevTools.
4. **Усно захистити роботу**, пояснивши логіку роботи CSS у власному проєкті.

4. Дослідницькі завдання

Перед виконанням практичної частини опрацюйте теоретичні матеріали та проведіть експерименти. Результати експериментів (спостереження та висновки) додайте до звіту.

4.1. Box Model

- Опрацювати CSS Box Model — web.dev.
- Створіть два блоки `<div>` з однаковими шириною та висотою. Одному задайте `box-sizing: content-box`, іншому — `border-box`. Додайте їм `padding` та `border`.
- Відкрийте DevTools (вкладка Computed). Поясніть у звіті, чому фактичний розмір блоків на екрані відрізняється, і зробіть висновок, який підхід є більш передбачуваним для верстки.

4.2. Cascade

- Опрацювати The Cascade — web.dev.
- Задайте колір тексту для одного елемента трьома способами з однаковою специфічністю (наприклад, через класи), але підключіть їх у різному порядку: через зовнішній файл, через тег `<style>` та змінюючи порядок оголошення класів у самому CSS.
- Яке правило «перемагає» і чому? Що відбувається, якщо змінити порядок підключення стилів?

4.3. Specificity

- Опрацювати Specificity — web.dev.
- Напишіть правила для одного елемента, використовуючи селектор тегу, селектор класу та селектор ID. Спробуйте «перебити» стиль ID за допомогою ланцюжка з кількох класів (наприклад, `.class1.class2.class3`…).
- Поясніть за допомогою калькуляції специфічності `(0,0,0)`, чому ID завжди перемагає будь-яку кількість класів. Продемонструйте це на скріншоті з DevTools.

4.4. Inheritance

- Опрацювати Inheritance — web.dev.
- Задайте `font-family` та `color` для тегу `body`. Розмістіть всередині текст у тегах `<p>`, `<a>` (посилання) та `<button>`.
- Яке правило успадкували стилі, а які — ні? Чому браузер не застосовує успадкування для певних тегів і як це виправити (наприклад, за допомогою значення `inherit`)?

4.5. CSS Variables

- Опрацювати матеріали про CSS Custom Properties на web.dev: у курсі Learn CSS вони зібрані в модулі Functions, розділ «Custom properties and `var()`». Докладніший опис області видимості та успадкування змінних — у MDN: Using CSS custom properties.
- Оголосіть кілька змінних (наприклад, кольори) у псевдокласі `:root`. Використайте їх у стилях. Потім спробуйте перевизначити ці змінні всередині конкретного класу (наприклад, `.dark-card`).
- Як працює локальна область видимості (scope) для CSS-змінних? Як це можна використати для створення тем або станів компонентів?

4.6. Повторне використання стилів і рефакторинг

- **Дослідження:** На основі попередніх тем, проаналізуйте концепцію створення перевикористовуваних утилітарних класів (наприклад, створення базового класу `.btn` та модифікаторів `.btn-primary`, `.btn-outline`).
- **Дослідницьке запитання:** Чому компонентний підхід до CSS кращий за написання унікальних стилів для кожного окремого елемента? Як це впливає на розмір CSS-файлу та його підтримку?

5. Практична частина

Завдання. Виконати верстку вебсторінки на основі запропонованого макету. Ви можете використовувати AI-інструменти (ChatGPT, Claude, GitHub Copilot тощо) для генерації базової структури HTML та CSS-стилів. Однак повна відповідальність за якість та архітектуру коду лежить на вас.

> **Важливо!** Практична реалізація макету виконується під час лабораторного заняття. У разі недостатньої кількості часу студент завершує роботу самостійно.

Вимоги до фінальної реалізації

У фінальному проєкті ви повинні свідомо застосувати досліджені принципи. Під час захисту ви маєте продемонструвати в коді та DevTools:

1. **Коректний Box Model.** Глобальне використання `box-sizing: border-box` та розуміння його впливу.
2. **Робота з inheritance.** Уникнення зайвого дублювання властивостей (шрифтів, кольорів), які можна успадкувати.
3. **Свідоме використання specificity.** Відсутність конструкцій `!important` (крім виняткових випадків утиліт) і надто довгих ланцюжків селекторів.
4. **CSS Variables.** Винесення кольорів, відступів і шрифтів у `:root` для легкого масштабування дизайну.

5. **Повторне використання класів.** Виділення спільних компонентів (наприклад, кнопок, карток, контейнерів) в окремі класи.
6. **Відсутність зайвого дублювання.** Код має бути DRY (Don't Repeat Yourself). Згенерований AI-код часто містить дублювання, тому потрібно провести рефакторинг.

Повний опис макета, структура HTML, набір CSS-змінних, точка адаптивності та критерії приймання винесені в окремий документ: Технічне завдання — секція тарифних планів.

6. Вимоги до звіту

Тема листа:

```
[Веб] ЛР-2: Основи CSS
```

У листі вказати посилання на:

```
GitHub repository: https://github.com/<username>/<repository>
GitHub Pages: https://<username>.github.io/<repository>/
Vercel: https://<project>.vercel.app/
```

> **Важливо!** Звітом до лабораторної роботи є виконаний і задеплоєний макет з розділу 5. Щоб захистити роботу, потрібно розуміти і пояснити:
>
> - чому елемент має саме такий розмір;
> - яке правило перемогло і чому;
> - чому властивість успадкувалася;
> - навіщо створена конкретна CSS Variable;
> - чому певний стиль винесений у спільний клас.

Обов'язково додавайте скріншоти панелі **DevTools** (вкладки Elements, Styles, Computed), які підтверджують ваші висновки.

7. Усний захист

Формат здачі лабораторної роботи передбачає усну співбесіду за результатами вашої практичної реалізації. Навіть якщо частину коду було згенеровано штучним інтелектом, ви повинні вміти пояснити кожен рядок.

Орієнтовні питання на захисті (із демонстрацією екрана):

- Відкрийте DevTools. Чому цей елемент має саме такий фізичний розмір (пояснити на основі Box Model)?

- Знайдіть у DevTools правило, яке було перекреслене. Яке правило перемогло і чому саме воно?
- Чому цей текстовий елемент отримав такий шрифт, хоча ви йому його не задавали? Покажіть ланцюжок успадкування.
- Навіщо в цьому конкретному місці ви використали CSS Variable? Що буде, якщо змінити її значення в `:root`?
- Покажіть приклад, де ви винесли стилі у спільний клас. Як би виглядав код без нього?

8. Оцінювання

1. Успішний захист — **2 бали**.
2. Додатковий **1 бал** за однієї з умов:
    - термін здачі до **23.09.2026**, АБО
    - робота на парі.